

# Primeira aplicação com Spring Boot MVC

## Visão geral do projeto GameTracker

Objetivo desta abertura: apresentar a arquitetura, o fluxo da aplicação e o papel de cada parte do projeto antes de entrar nos decks específicos de Model, Repository, Service, Controller e View/Thymeleaf.

**Projeto base  
CRUD web completo**

### Tecnologias

Java 17, Spring Boot, MVC, Spring Data JPA, Thymeleaf, H2 e Validation.

### Domínio do exemplo

Cadastro de jogos com status: quero jogar, jogando e zerado.

### Estratégia da aula

Começar pelo panorama geral e depois aprofundar cada camada separadamente.

# O que a aplicação faz

O projeto é simples, mas cobre o ciclo completo de uma aplicação web MVC.

- listar jogos cadastrados na aplicação
- abrir formulário para novo cadastro
- salvar dados validados no banco H2
- editar um jogo já existente
- excluir registros da lista
- renderizar páginas HTML com Thymeleaf

## Por que este projeto é bom para iniciar?

Porque ele mostra requisição, controller, service, repository, banco e view funcionando juntos.

## O aluno aprende o quê?

Como os arquivos se conectam e como uma ação no navegador vira persistência e resposta HTML.

## Resumo em uma frase

É um CRUD web completo que materializa a arquitetura MVC sem complicar demais o domínio.

# Arquitetura MVC do projeto

Antes de ler o código, vale fixar quem faz o quê.



## Ideia central

Cada camada tem responsabilidade própria. O ganho não é só “organização bonita”; é manutenção, teste e evolução sem bagunçar o projeto inteiro.

# Fluxo de uma requisição

Este é o caminho percorrido quando o usuário abre a listagem ou envia um formulário.

- 1 Navegador acessa /games
- 2 Controller recebe a rota
- 3 Service executa a lógica
- 4 Repository consulta ou grava no banco
- 5 Dados voltam ao Controller
- 6 Controller envia os dados para a View
- 7 Thymeleaf gera o HTML final

## Exemplo real do projeto

### GET /games

Controller chama `gameService.listarTodos()`  
Service usa `gameRepository.findAll()`  
Controller coloca os dados no Model  
Retorna a view “lista”

### Leitura para o aluno

O navegador não “fala” com o banco. Ele fala com o Controller.

# Estrutura do projeto

A organização em pacotes e recursos facilita leitura e manutenção.

```
src/main/java/com/exemplo/gametracker
├── GameTrackerApplication.java
├── controller/GameController.java
├── model/Game.java
├── repository/GameRepository.java
└── service/GameService.java

src/main/resources
├── templates/lista.html
├── templates/formulario.html
└── static/css/style.css
```

## Java

Contém classes de domínio, regras, acesso a dados e rotas.

## templates

Páginas HTML processadas pelo Thymeleaf.

## static/css

Arquivos estáticos usados na interface.

# Trecho de código que amarra o projeto

A classe principal inicia a aplicação e ativa a autoconfiguração do Spring Boot.

```
1 @SpringBootApplication
2 public class GameTrackerApplication {
3
4     public static void main(String[] args) {
5
6         SpringApplication.run(GameTrackerApplication.class,
7                                args);
8     }
9 }
```

- o método main sobe a aplicação
- o Spring Boot configura servidor e contexto
- o escaneamento encontra Controller, Service e Repository

## Mensagem didática

Sem essa classe, o projeto não inicializa. É a porta de entrada da aplicação.

# Mapa da sequência didática

Depois desta abertura, cada parte pode ser aprofundada em um deck específico.

## Parte 1 — Model

entidade, atributos, JPA e validação

## Parte 2 — Repository

persistência com JpaRepository

## Parte 3 — Service

regras de negócio e intermediação

## Parte 4 — Controller

rotas, formulários e redirecionamentos

## Parte 5 — View / Thymeleaf

templates, bindings e renderização HTML

## Estratégia

Primeiro panorama. Depois profundidade. É o oposto do caos.

# Encerramento da abertura

O aluno já deve sair desta parte entendendo o panorama antes do detalhamento técnico.

- o projeto é um CRUD web completo com Spring Boot MVC
- cada camada tem uma responsabilidade bem definida
- o fluxo completo vai do navegador até o banco e volta para o HTML
- os próximos decks aprofundam cada parte separadamente

## Próximo passo em sala

Abrir o deck da Parte 1 e começar pelo Model. A partir daí, mostrar como cada arquivo entra no fluxo geral apresentado aqui.

### Gancho final

Agora vamos sair da visão geral e entrar no código.



## Parte 1 — Model

Entidade, mapeamento JPA e validação  
no projeto GameTracker MVC

Vamos isolar a primeira camada do sistema e entender por  
que a classe Game é o ponto de partida de todo o fluxo.

---

Projeto da aula: GameTracker MVC

*Primeiro entendemos o dado. Depois o Spring faz magia — mas com método.*

### Game.java

@Entity

id : Long

nome : String

plataforma : String

genero : String

status : String

@NotBlank nos campos textuais

# O que vamos dominar nesta etapa

Foco total no model: representar dados, mapear para o banco e validar entrada.

- Compreender o papel do Model dentro do fluxo MVC.
- Ler a classe Game como contrato de dados da aplicação.
- Interpretar anotações JPA e Validation sem decorar no escuro.
- Relacionar objeto Java, tabela do banco e formulário web.

## Sequência da disciplina

1 Model

2 Repository

3 Service

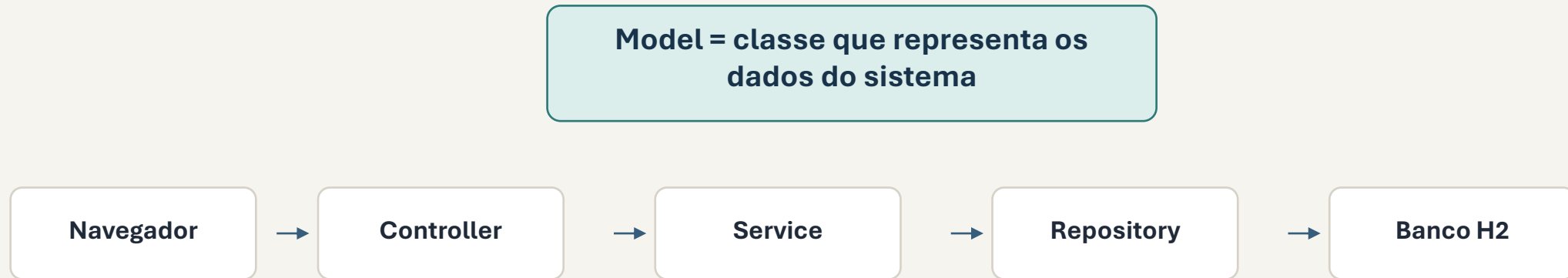
4 Controller

5 View

Hoje o objetivo é sair sabendo por que a classe de domínio é a base das demais camadas.

# Onde o Model entra no MVC?

Observe o fluxo completo, mas note que hoje o nosso zoom está na estrutura do dado.



No GameTracker, esse papel é cumprido principalmente por `Game`.

*Sem Model não há estrutura para persistir, validar ou trafegar dados entre as camadas.*

# O que a entidade Game representa?

A classe descreve um jogo cadastrado e define quais informações o sistema considera relevantes.

- `id` identifica o registro e nasce do banco.
- `nome`, `plataforma`, `genero` e `status` compõem o estado de negócio.
- A entidade vira ponte entre formulário, regras e persistência.
- É aqui que o projeto deixa de ser “genérico” e ganha domínio.

```
com.exemplo.gametracker.model.Game
```

```
id : Long
```

```
nome : String
```

```
plataforma : String
```

```
genero : String
```

```
status : String
```

Entidade JPA

Validação

Domínio

# Estrutura essencial da classe Game.java

Nesta primeira aula, vale mais enxergar a anatomia da entidade do que decorar todos os getters e setters.

```
1 package com.exemplo.gametracker.model;
2
3 @Entity
4 @Table(name = "game")
5 public class Game {
6
7     @Id
8     @GeneratedValue(strategy = GenerationType.IDENTITY)
9     private Long id;
10
11     @NotBlank(message = "O nome do jogo é obrigatório")
12     private String nome;
13
14     @NotBlank(message = "A plataforma é obrigatória")
15     private String plataforma;
16
17     @NotBlank(message = "O gênero é obrigatório")
18     private String genero;
19
20     @NotBlank(message = "O status é obrigatório")
21     private String status;
22
23     // getters e setters
24 }
```

## Leitura pedagógica

- A classe nasce no pacote `model`.
- O bloco de atributos concentra o estado do domínio.
- As anotações são metadados que o Spring/JPA interpretam.
- Getters e setters viabilizam binding e acesso ao estado.

# Leitura guiada do que importa na entidade

As anotações fazem a classe conversar com o JPA e com a validação do Spring.

```
1 @Entity
2 @Table(name = "game")
3 public class Game {
4
5     @Id
6     @GeneratedValue(strategy =
GenerationType.IDENTITY)
7     private Long id;
8
9     @NotBlank(message = "O nome do
jogo é obrigatório")
10    private String nome;
11 }
```

**@Entity**

Marca a classe como entidade gerenciada pelo JPA.

**@Table(name = "game")**

Define explicitamente o nome da tabela no banco.

**@Id**

Indica qual campo funciona como chave primária.

**@GeneratedValue (IDENTITY)**

Delega a geração do id ao banco.

**@NotBlank**

Rejeita texto vazio ou composto apenas por espaços.

# Como o objeto vira tabela

É o mesmo dado com duas roupas: objeto em Java e registro relacional no banco.

## Objeto Java Game

- id
- nome
- plataforma
- genero
- status



## Tabela: game

| coluna     | tipo    | observação |
|------------|---------|------------|
| id         | BIGINT  | PK         |
| nome       | VARCHAR | NOT NULL*  |
| plataforma | VARCHAR | NOT NULL*  |
| genero     | VARCHAR | NOT NULL*  |
| status     | VARCHAR | NOT NULL*  |

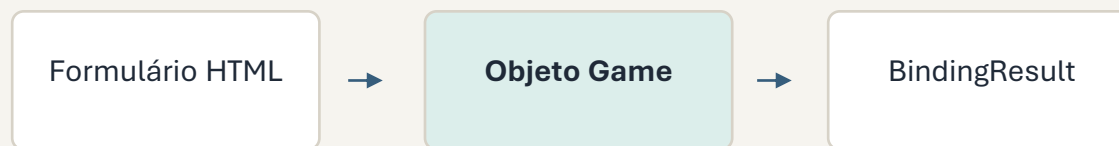
*\* A restrição exata no banco depende da estratégia de schema e da configuração JPA. Em aula, o importante é entender a intenção semântica do Model.*

# Validação começa no Model, não no susto

Os campos já carregam as regras mínimas que o formulário precisa respeitar.

```
1 @NotBlank
2 private String nome;
3
4 @NotBlank
5 private String plataforma;
6
7 @NotBlank
8 private String genero;
9
10 @NotBlank
11 private String status;
```

```
1 salvar(@Valid @ModelAttribute("game")
2       Game game,
3       BindingResult result, ...)
```



Se algum campo vier vazio, a própria entidade já sabe qual mensagem precisa ser exibida. Isso mantém a regra perto do dado.



# Onde a classe Game aparece nas próximas camadas

A mesma entidade reaparece do banco até a tela — por isso o desenho inicial precisa ser sólido.

## Repository

```
1 public interface
GameRepository
2     extends
JpaRepository<Game,
Long> {}
```

- Usa `Game` como tipo da entidade.
- Define `Long` como tipo do identificador.

## Service

```
1 public void
salvar(Game game) {
2     gameRepository.save(game
);
3 }
```

- Recebe `Game` como argumento de negócio.
- Encaminha a entidade para persistência.

## Controller

```
1
model.addAttribute("game
", new Game());
```

- Cria ou envia um objeto `Game` para a view.
- O formulário se liga aos campos da entidade.

*Leitura pedagógica: a entidade nasce no model e reaparece em praticamente toda a aplicação.*

# Evolua o Model: adicionar nota de 0 a 10

Exercício curto para a turma consolidar JPA + Validation sem ainda depender de todas as telas prontas.

## Passo a passo

- Criar o campo `private Integer nota;` na classe Game.
- Anotar com `@Min(0)` e `@Max(10)` para limitar a faixa válida.
- Gerar getter e setter do novo atributo.
- Discutir em dupla: quais outras camadas precisarão mudar depois?

```
1 import jakarta.validation.constraints.Max;  
2 import jakarta.validation.constraints.Min;  
3  
4 @Min(0)  
5 @Max(10)  
6 private Integer nota;
```

## Discussão final

- formulário precisará do novo campo
- lista poderá exibir a nota
- persistência herdar a mudança pelo mesmo model

*Model bem pensado economiza retrabalho no resto do MVC.*

## Parte 2 — Repository

Como criar a camada que conversa com o banco sem transformar a aula em um festival de SQL manual

Objetivo pedagógico: ensinar o passo a passo da implementação e mostrar a evolução até o uso do Spring Data JPA.

---

Projeto da aula: GameTracker MVC

*Aqui o aluno percebe que “persistência” não precisa nascer como sofrimento artesanal.*

### GameRepository. java

package repository

interface GameRepository

extends JpaRepository<Game,  
Long>

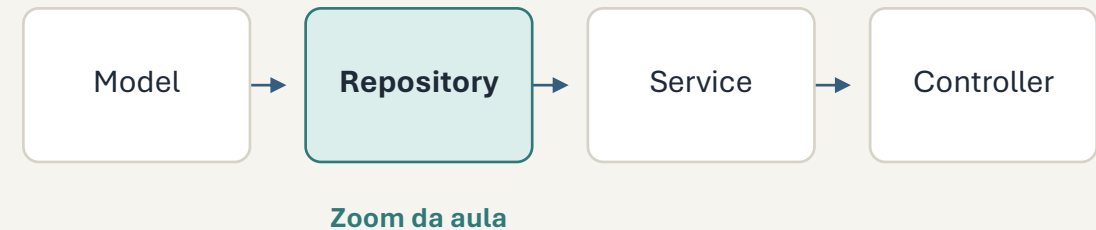
save / findAll / findById / delete

implementação quase zero, poder alto

# Onde o Repository se encaixa no projeto

Ele é a camada responsável por persistir e recuperar entidades sem jogar detalhes de banco para o restante da aplicação.

- Entender o papel do Repository na arquitetura MVC + Service.
- Aprender o passo a passo para criar ``GameRepository``.
- Ler os genéricos de ``JpaRepository<Game, Long>`` com confiança.
- Relacionar métodos herdados com o CRUD da aplicação.

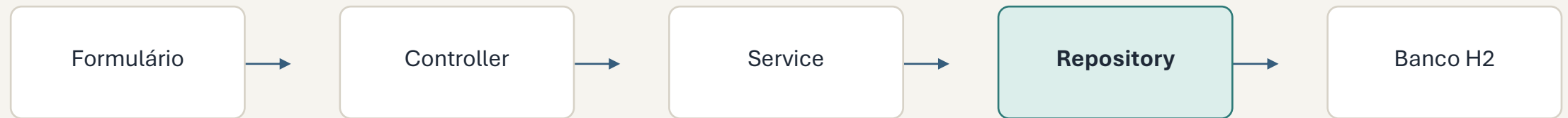


## Ideia-chave

O Controller pede, o Service organiza e o Repository executa o acesso aos dados.

# O que é um Repository, na prática?

Pense nele como a porta oficial entre o modelo de domínio e o banco de dados.



## O Repository recebe

Entidades, ids e pedidos de consulta.

## O Repository devolve

Objetos Java já prontos para o Service usar.

## O ganho didático

Separar regra de negócio de acesso a dados.

# Como criar o Repository do zero

Esta é a sequência que o aluno consegue repetir sozinho depois da aula.

## PASSO 1

### Criar pacote

Adicione ``repository``  
ao lado de ``model``,  
``service`` e  
``controller``.



## PASSO 2

### Criar interface

Declare ``public``  
interface  
`GameRepository``.  
Nada de classe  
concreta aqui.



## PASSO 3

### Estender JPA

Use ``extends``  
`JpaRepository<Game,`  
`Long>`` para herdar o  
CRUD.



## PASSO 4

### Injetar no Service

Receba o repository no  
construtor e use  
``findAll``, ``save``,  
``findById``, ``delete``.

# Como a solução amadurece até chegar no Spring Data JPA

Mostre a lógica da evolução: primeiro entendemos o problema, depois reduzimos código sem perder arquitetura.

## Fase 1 — memória

```
1 // fase inicial didática
2 private final List<Game>
jogos = new ArrayList<>();
3
4 public List<Game>
listarTodos() {
5     return jogos;
6 }
7
8 public void salvar(Game
game) {
9     jogos.add(game);
10 }
```

funciona

## Fase 2 — contrato

```
1 // contrato pensado para
persistência
2 public interface
GameRepository {
3     List<Game> findAll();
4     Optional<Game>
findById(Long id);
5     Game save(Game game);
6     void delete(Game game);
7 }
```

organiza

## Fase 3 — Spring Data

```
1 // versão final do projeto
2 public interface
GameRepository
3     extends
JpaRepository<Game, Long> {
4 }
```

escala

*Mensagem para a turma: a evolução não elimina conceitos; ela elimina trabalho repetitivo.*

# Leitura guiada de GameRepository.java

Poucas linhas, mas cada uma resolve um problema importante.

## Arquivo real do projeto

```
1 package com.exemplo.gametracker.repository;
2
3 import com.exemplo.gametracker.model.Game;
4 import
org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface GameRepository extends
JpaRepository<Game, Long> {
7 }
```

## Como explicar em sala

- `interface` : define contrato, não comportamento manual.
- `JpaRepository<Game, Long>` : o primeiro tipo é a entidade; o segundo, o tipo do id.
- Ao estender a interface do Spring, herdamos operações CRUD prontas.
- Se precisar, podemos adicionar métodos de consulta depois.



# Quais métodos do Repository aparecem no Service

Aqui o aluno percebe que o CRUD da aplicação conversa diretamente com a API herdada do JPA.

## Métodos herdados do JpaRepository

**findAll()**      listar todos os jogos

**findById(id)**      buscar um jogo por id

**save(game)**      salvar novo jogo

**save(gameExistente)**      persistir atualização

**delete(game)**      remover registro

## Trecho do GameService

```
1 public List<Game> listarTodos() {  
2     return gameRepository.findAll();  
3 }  
4  
5 public void salvar(Game game) {  
6     gameRepository.save(game);  
7 }  
8  
9 public void excluir(Long id) {  
10     Game game = buscarPorId(id);  
11     gameRepository.delete(game);  
12 }
```

Repare: o Service continua mandando na regra; o Repository continua mandando no acesso aos dados.

# Quando o CRUD básico não basta

Depois que a turma domina o essencial, você pode mostrar que o Repository também permite consultas derivadas.

## Exemplo de próxima aula

```
1 public interface GameRepository
2     extends JpaRepository<Game, Long> {
3
4     List<Game> findByStatus(String status);
5
6     List<Game> findByPlataforma(String
7     plataforma);
8 }
```

- O nome do método vira a consulta.
- Spring Data interpreta `findBy...` automaticamente.
- Isso mantém o código expressivo e curto.
- Excelente gancho para filtros e busca na interface web.

### Observação didática

Esses métodos não estão no projeto atual; são a continuação natural depois do CRUD funcionar.

# Erros comuns ao criar o Repository

Se a aplicação não sobe, normalmente o problema mora em um detalhe pequeno.

## Erros frequentes

- Criar `GameRepository` como classe em vez de interface.
- Esquecer de importar `JpaRepository` do pacote correto.
- Usar tipo de id diferente do definido na entidade.
- Colocar o arquivo fora do pacote escaneado pelo Spring.
- Tentar colocar regra de negócio pesada dentro do repository.

## Antes de rodar, confirme

- A entidade `Game` existe e está anotada com `@Entity`.
- O projeto tem Spring Data JPA nas dependências.
- O `GameService` recebe `GameRepository` no construtor.
- O banco está configurado e a aplicação inicia sem exceções.

# Prática de 10–15 minutos para fixar a camada Repository

A ideia é repetir a receita do projeto, mas com pequenas decisões feitas pela turma.

## Enunciado

1. Criar a interface `GameRepository``.
2. Estender `JpaRepository<Game, Long>``.
3. Injetar no `GameService``.
4. Executar a aplicação e testar cadastro + listagem.
5. Como desafio extra, propor `findByStatus(String status)``.

## Critério de sucesso

A aplicação lista os jogos normalmente e o código do service compila usando o repository.

### Pergunta para a sala:

Por que o Repository deve conhecer o banco, mas o Controller não?

## Fechamento do professor

Resumo que vale ouro:

- Model define o dado.
- Repository persiste o dado.
- Service organiza o uso do dado.
- Controller expõe o fluxo para a web.

*Se essa cadeia ficou clara, a turma já saiu da fase “copiar código” para a fase “entender arquitetura”.*

## Parte 3 — Service

Como construir a camada que organiza a lógica sem entupir o Controller de responsabilidades

O passo a passo da implementação e mostrar a evolução do Service, do repasse simples até a regra de negócio.

---

Projeto da aula: GameTracker MVC

*Se o Controller começa a fazer tudo, o caos faz sprint e chega antes do prazo.*

### GameService.java

```
@Service
```

```
listarTodos()
```

```
buscarPorId(id)
```

```
salvar(game)
```

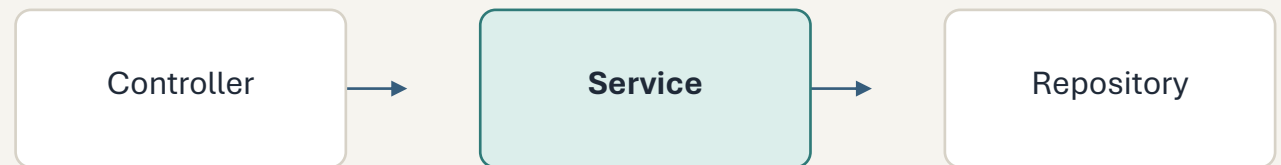
```
atualizar(id, game)
```

```
excluir(id)
```

# Onde o Service se encaixa no projeto

Ele fica entre Controller e Repository, organizando a lógica da aplicação e mantendo cada camada no seu papel.

- Entender por que o Service existe em uma aplicação MVC.
- Aprender o passo a passo para criar ``GameService``.
- Relacionar métodos do Service com o CRUD do projeto.
- Mostrar a evolução da implementação até a versão final.

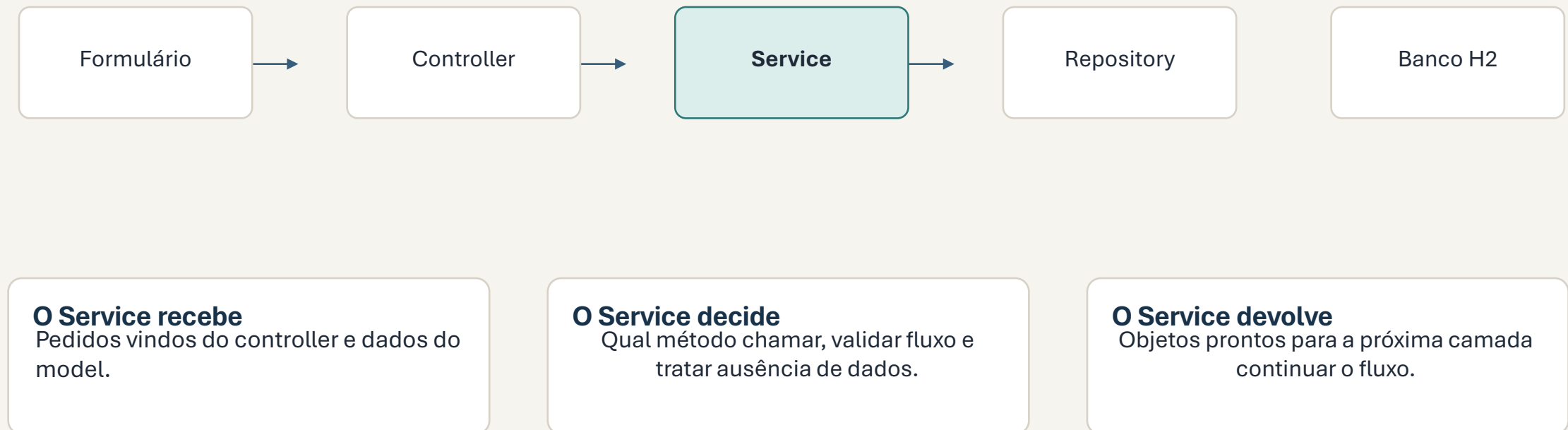


## Ideia-chave

O Controller lida com HTTP. O Repository lida com persistência. O Service lida com a regra e com a coordenação entre os dois.

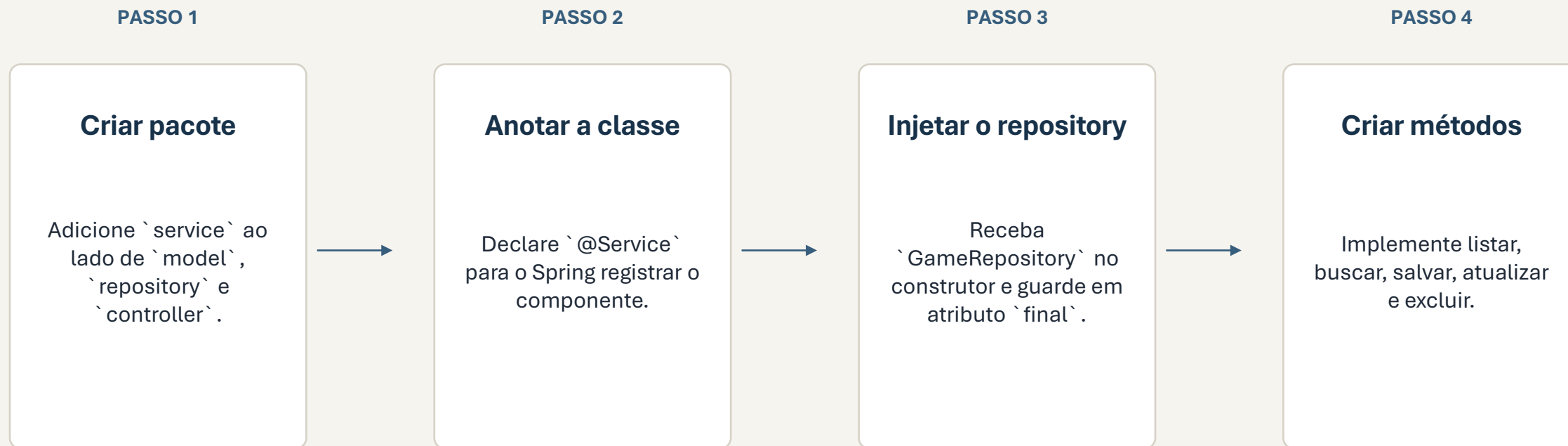
# O que o Service faz, na prática?

Nem tudo é “regra de negócio épica”. Às vezes ele apenas organiza chamadas — e isso já é arquitetura.



# Como criar o Service do zero

Essa sequência é a receita que a turma consegue repetir sem colar no colega nem no Stack Overflow.





# Como o Service amadurece durante a construção

A lógica cresce em etapas: primeiro separar camadas, depois centralizar decisões, depois tratar casos reais.

## Fase 1 — controller pesado

```
1 // fase inicial (não ideal)
2 @PostMapping("/games")
3 public String salvar(Game
  game) {
4
5   gameRepository.save(game);
6   return "redirect:/games";
7 }
```

separa

## Fase 2 — service básico

```
1 // fase intermediária
2 public List<Game>
  listarTodos() {
3   return
4   gameRepository.findAll();
5 }
6 public void salvar(Game game)
  {
7
8   gameRepository.save(game);
9 }
```

organiza

## Fase 3 — service de verdade

```
1 // fase final do projeto
2 public Game buscarPorId(Long
  id) {
3   return
4   gameRepository.findById(id)
5   .orElseThrow(() ->
6   new IllegalArgumentException(
7   "Jogo não
8   encontrado para o id " + id));
9 }
10 public void atualizar(Long
    id, Game gameAtualizado) {
11   // regra de atualização
12 }
```

coordena

*Mensagem para a turma: o Service não nasce “grande”; ele nasce útil e cresce conforme o fluxo exige.*

# Leitura guiada de GameService.java

Agora mostramos o arquivo real do projeto e explicamos como cada método participa do CRUD.

## Arquivo real do projeto

```
1 package com.exemplo.gametracker.service;
2
3 import com.exemplo.gametracker.model.Game;
4 import com.exemplo.gametracker.repository.GameRepository;
5 import org.springframework.stereotype.Service;
6
7 import java.util.List;
8
9 @Service
10 public class GameService {
11
12     private final GameRepository gameRepository;
13
14     public GameService(GameRepository gameRepository) {
15         this.gameRepository = gameRepository;
16     }
17
18     public List<Game> listarTodos() {
19         return gameRepository.findAll();
20     }
21 }
```

## Como explicar em sala

- `@Service` marca a camada para o Spring gerenciar.
- A injeção por construtor deixa a dependência explícita.
- `buscarPorId` evita repetição de busca e tratamento de ausência.
- `atualizar` muda campos específicos antes de salvar.
- `excluir` reutiliza a busca antes da remoção.

# Como o Controller conversa com o Service

Nesta leitura, a turma enxerga o Service como ponto de passagem inteligente entre HTTP e persistência.

## Trecho do GameController

```

1 @GetMapping("/games")
2 public String listar(Model model) {
3     model.addAttribute("games",
gameService.listarTodos());
4     return "lista";
5 }
6
7 @PostMapping("/games")
8 public String salvar(@Valid
@ModelAttribute("game") Game game,
9                     BindingResult result,
10                    Model model,
11                    RedirectAttributes
redirectAttributes) {
12     if (result.hasErrors()) {
13         model.addAttribute("modoEdicao",
false);

```

```

14         return "formulario";

```

15 Leitura didática: o Controller chama o Service; o Service chama o Repository. Cada um fica no seu quadrado — e a aplicação agradece.

## Primeiros métodos do Service

```

1 @Service
2 public class GameService {
3
4     private final GameRepository
gameRepository;
5
6     public GameService(GameRepository
gameRepository) {
7         this.gameRepository = gameRepository;
8     }
9
10    public List<Game> listarTodos() {
11        return gameRepository.findAll();
12    }
13
14    public void salvar(Game game) {
15        gameRepository.save(game);

```

```

16

```

# Por que o método atualizar merece destaque

É aqui que o aluno entende que o Service pode fazer mais do que apenas repassar chamadas.

## Trecho central do método atualizar

```
1 public void atualizar(Long id, Game  
gameAtualizado) {  
2     Game gameExistente = buscarPorId(id);  
3  
4  
gameExistente.setNome(gameAtualizado.getNome());  
5  
gameExistente.setPlataforma(gameAtualizado.getPlataf  
orma());  
6  
gameExistente.setGenero(gameAtualizado.getGenero());  
7  
gameExistente.setStatus(gameAtualizado.getStatus());  
8  
9     gameRepository.save(gameExistente);  
10
```

## O que acontece nessa lógica

- Busca o registro existente antes de alterar.
- Evita criar um objeto “solto” sem contexto do banco.
- Atualiza apenas os campos desejados.
- Salva novamente o objeto já carregado.

*Gancho para discussão: depois podemos validar status, impedir edições indevidas ou aplicar regras antes do save.*

# Erros comuns ao criar a camada Service

Quase sempre o problema não é “o Spring odeia você”; é só um detalhe escapando no código.

## Erros frequentes

- Colocar lógica de negócio direto no Controller.
- Esquecer a anotação `@Service`.
- Não injetar o `GameRepository` pelo construtor.
- Duplicar busca por id em vários lugares sem centralizar.
- Atualizar entidade sem carregar o registro existente.

## Antes de rodar, confirme

- O pacote `service` está dentro da estrutura escaneada pelo Spring.
- A classe possui `@Service`.
- O construtor recebe `GameRepository`.
- O Controller chama `gameService`, não `gameRepository`.
- Os métodos principais do CRUD existem e compilam.

# Prática de 10–15 minutos para fixar o Service

A meta é fazer a turma construir a camada e perceber o valor da separação de responsabilidades.

## Enunciado

1. Criar a classe ``GameService``.
2. Anotar com ``@Service``.
3. Injetar ``GameRepository`` no construtor.
4. Implementar ``listarTodos``, ``salvar``, ``buscarPorId``, ``excluir``.
5. Como desafio extra, implementar ``atualizar``.

## Critério de sucesso

O Controller compila usando ``gameService`` e o fluxo de cadastro/listagem continua funcionando.

### Pergunta para a sala:

Se o Service sumisse agora, o que iria parar no Controller?

## Fechamento do professor

Resumo que fixa:

- Controller recebe a requisição.
- Service organiza a lógica.
- Repository persiste.
- View exibe o resultado.

*Quando a turma entende esse papel, o MVC deixa de ser sigla bonita e vira ferramenta real.*

## Parte 4

# Controller no Spring Boot MVC

Como mapear rotas, receber dados do formulário, validar, redirecionar e completar o CRUD web.

```
@GetMapping  
@PostMapping  
@PathVariable  
@ModelAttribute
```

**Controller**

recebe requisições  
chama o service  
retorna view ou redirect

# Qual é o papel do Controller?

Conceito

Ele é a borda web da aplicação, não o depósito de toda a lógica.

## Receber a requisição

URL, formulário e parâmetros entram aqui.

## Delegar para o service

O controller coordena; o service decide a regra.

## Responder com view ou redirect

Model para tela, redirect para fluxo pós-POST.



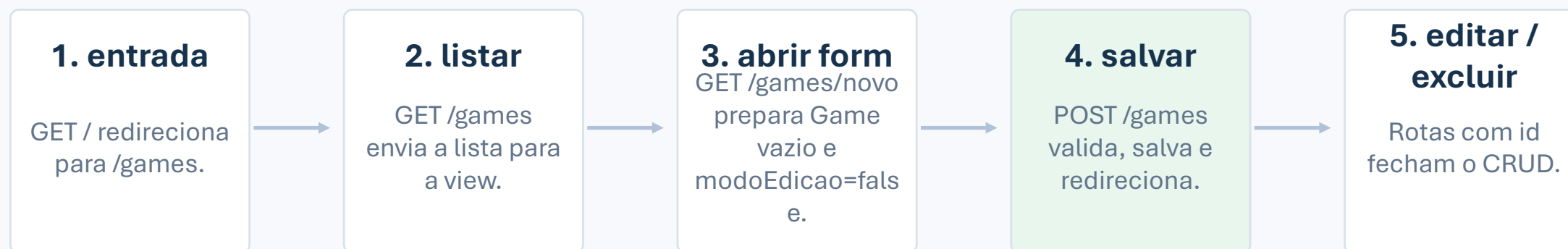
Moral da história: controller é maestro, não banda inteira.



# Evolução da implementação do Controller

Evolução

Ensine na ordem em que a aplicação vai ganhando vida.



# Estrutura da classe GameController

@Controller + dependência do service + métodos mapeados.

GameController.java

```
1 @Controller
2 public class GameController {
3
4     private final GameService gameService;
5
6     public GameController(GameService gameService) {
7         this.gameService = gameService;
8     }
9
10    // métodos com @GetMapping e @PostMapping
11 }
```

Classe

## @Controller

Indica ao Spring que a classe atende requisições MVC e retorna views.

## private final GameService

O controller depende da camada de serviço para não misturar responsabilidades.

## Injeção via construtor

Didática e limpa: deixa explícito do que a classe precisa para funcionar.

# Rotas de navegação: início, listagem e novo formulário

GET

Primeiro objetivo em sala: mapear as URLs mais simples.

## Rotas GET

```
1 @GetMapping("/")
2 public String inicio() {
3     return "redirect:/games";
4 }
5
6 @GetMapping("/games")
7 public String listar(Model model) {
8     model.addAttribute("games",
gameService.listarTodos());
9     return "lista";
10 }
11
12 @GetMapping("/games/novo")
13 public String abrirFormularioNovo(Model model) {
14     model.addAttribute("game", new Game());
15     model.addAttribute("modoEdicao", false);
16     return "formulario";
17 }
```

### redirect:/games

Leva o usuário direto para o fluxo principal.

### Model model

Envia a chave "games" para a view de listagem.

### new Game() + modoEdicao=false

Prepara o binding do formulário e a lógica visual da view.

# POST /games: recebendo dados do formulário

[Salvar](#)

É aqui que o controller começa a lidar com entrada, validação e redirect.

Salvar com validação

```
1 @PostMapping("/games")
2 public String salvar(@Valid @ModelAttribute("game") Game
game,
3                     BindingResult result,
4                     Model model,
5                     RedirectAttributes redirectAttributes)
{
6
7     if (result.hasErrors()) {
8         model.addAttribute("modoEdicao", false);
9         return "formulario";
10    }
11
12    gameService.salvar(game);
13    redirectAttributes.addFlashAttribute("mensagem",
14        "Jogo cadastrado com sucesso.");
15    return "redirect:/games";
16 }
```

## @Valid + BindingResult

Se a entidade vier inválida, o controller detecta e volta para a tela.

## @ModelAttribute("game")

Liga o formulário HTML ao objeto Game.

## RedirectAttributes

Mensagem de sucesso sem quebrar o padrão Post/Redirect/Get.

## gameService.salvar(game)

Persistência fica no service; o controller só orquestra.

# Edição: o mesmo formulário serve para cadastrar e atualizar

[Editar](#)

Fluxo de edição

```
1 @GetMapping("/games/{id}/editar")
2 public String abrirFormularioEdicao(@PathVariable Long id, Model
model) {
3     model.addAttribute("game", gameService.buscarPorId(id));
4     model.addAttribute("modoEdicao", true);
5     return "formulario";
6 }
7
8 @PostMapping("/games/{id}/editar")
9 public String atualizar(@PathVariable Long id,
10                        @Valid @ModelAttribute("game") Game game,
11                        BindingResult result, Model model,
12                        RedirectAttributes redirectAttributes) {
13     if (result.hasErrors()) {
14         model.addAttribute("modoEdicao", true);
15         return "formulario";
16     }
17     gameService.atualizar(id, game);
18     return "redirect:/games";
19 }
```

## @PathVariable Long id

O id vem da URL e identifica qual jogo será alterado.

## modoEdicao = true

A view muda rótulos, ação do form e comportamento.

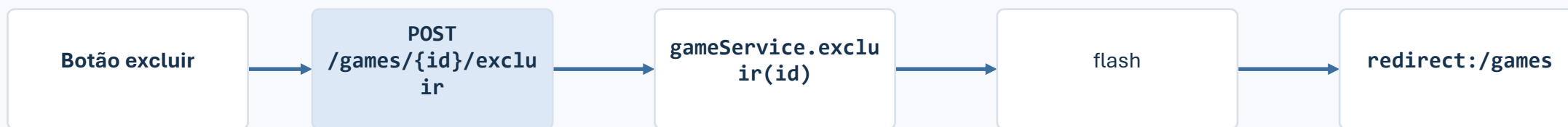
## Reaproveitamento com propósito

Perfeito para aula: o aluno percebe que controller e view trabalham juntos.

# Exclusão e fluxo completo do CRUD

[Excluir](#)

Recebe o id, delega a exclusão e volta para a listagem.



## Método de exclusão

```
1 @PostMapping("/games/{id}/excluir")
2 public String excluir(@PathVariable Long id,
3                       RedirectAttributes
4                       redirectAttributes) {
5     gameService.excluir(id);
6     redirectAttributes.addFlashAttribute("mensagem",
7     "Jogo excluído com sucesso.");
8     return "redirect:/games";
9 }
```

# Roteiro de implementação em sala

Sequência curta para live coding sem transformar o projecter em boss final.

## Passo 1

Implemente /, /games e /games/novo. Faça a navegação funcionar.

## Passo 2

Implemente POST /games com validação e redirect.

## Passo 3

Adicione editar e excluir para fechar o CRUD.

## Checklist de explicação

- qual URL chama cada método
- quais parâmetros entram em cada método
- quando retornar view e quando retornar redirect
- por que o controller chama o service, e não o repository diretamente
- como o mesmo formulário atende cadastro e edição

## Parte 5

# View / Thymeleaf no Spring Boot MVC

Como transformar dados do Model em páginas HTML dinâmicas, com formulário, listagem, validação e CSS.

### Arquivos desta parte

#### lista.html

Renderiza a tabela de jogos, o botão de novo cadastro, a mensagem de sucesso e o estado vazio da tela.

#### formulario.html

Usa th:object, th:field e ação dinâmica para salvar ou editar o mesmo objeto Game.

#### style.css

Separa a apresentação visual da lógica do template e melhora a experiência de uso.

#### Objetivo didático

Mostrar não só o que a View faz, mas a sequência de construção até a página final.



# Qual é o papel da View?

Ela recebe dados preparados pelo Controller e transforma isso em HTML para o navegador.

## Exibir dados

Exemplo: a coleção games vira linhas da tabela em lista.html.

## Receber entrada

O formulário captura nome, plataforma, gênero e status.

## Renderizar feedback

Mensagens de erro e sucesso aparecem na própria interface.

## Apoiar navegação

Links e ações apontam para as rotas do Controller.

*Moral da história: a View não consulta banco nem decide regra de negócio. Ela apresenta.*

## Thymeleaf no centro da renderização

**th:text** substitui o conteúdo do elemento por um valor do Model

**th:each** repete um bloco para cada item da coleção

**th:object** define o objeto-base usado no formulário

**th:field** faz o binding entre campo HTML e atributo do objeto

**th:if** controla a renderização condicional

**th:href** / **th:action** gera URLs alinhadas às rotas da aplicação

# Evolução da implementação da View

Ensine na ordem em que o aluno percebe valor: primeiro enxerga a tela, depois entende o motor por trás.

## 1 HTML base

Criar a estrutura HTML com título, card e áreas da página.

## 2 Lista dinâmica

Trocar conteúdo fixo por `th:each`, `th:text` e links com `th:href`.

## 3 Formulário

Ligar campos ao objeto Game com `th:object` e `th:field`.

## 4 Validação

Exibir erros de campo e mensagens de sucesso na interface.

## 5 CSS e polimento

Aplicar `style.css` para entregar uma aplicação com cara de projeto real.

Estratégia de aula: mostre a página pronta rapidamente, volte uma camada e reconstrua a tela em blocos. O aluno entende melhor quando liga cada atributo do Thymeleaf ao efeito visual que ele produz.

## Estrutura em src/main/resources

resources

```
1 src/main/resources
2 |   static
3 |     |   css
4 |     |     style.css
5 |   templates
6 |     |   formulario.html
7 |     |   lista.html
```

### templates

Arquivos HTML processados pelo Thymeleaf. O retorno do Controller aponta para esses nomes.

### static

Arquivos estáticos servidos diretamente pelo Spring Boot, como CSS, JS e imagens.

### Por que separar?

Template cuida da estrutura dinâmica. CSS cuida da aparência. Misturar tudo vira gambiarra gourmet.

### carregamento do CSS

```
1 <link rel="stylesheet"
  th:href="@{/css/style.css}">
```

### retornos do Controller

```
1 return "lista";
2 return "formulario";
```

### Leitura didática

- quando o método retorna "lista", o Spring procura templates/lista.html
- o CSS em static/css fica acessível pela URL /css/style.css
- essa organização aproxima a prática de um projeto real

## lista.html: exibindo a coleção de games

lista.html

```

1 <a th:href="@{/games/novo}" class="botao-principal">Novo jogo</a>
2
3 <div th:if="{mensagem}" class="mensagem-sucesso"
4   th:text="{mensagem}"></div>
5
6 <tr th:each="game : {games}">
7   <td th:text="{game.nome}"></td>
8   <td th:text="{game.plataforma}"></td>
9   <td th:text="{game.genero}"></td>
10  <td><span class="badge" th:text="{game.status}"></span></td>
11  <td class="coluna-acoes">
12    <a
th:href="@{/games/{id}/editar(id={game.id})}">Editar</a>
13    <form th:action="@{/games/{id}/excluir(id={game.id})}"
method="post">
14      <button type="submit" class="link-
botao">Excluir</button>
15    </form>
16  </td>
17 </tr>
18
19 <tr th:if="{#lists.isEmpty(games)}">
20   <td colspan="5" class="linha-vazia">Nenhum jogo cadastrado
ainda.</td>
21 </tr>

```

### th:each

Percorre a coleção enviada pelo Controller em `model.addAttribute("games", ...)`.

### th:text

Preenche as células da tabela com os atributos de cada objeto Game.

### th:href / th:action

Gera URLs de editar e excluir usando o id do registro atual.

### Estado vazio e flash message

A View já trata lista vazia e feedback de sucesso, sem colocar HTML no Controller.

## formulario.html: binding com o objeto Game

formulario.html

```
1 <h2 th:text="${modoEdicao} ? 'Editar jogo' : 'Novo
jogo' "></h2>
2
3 <form th:action="${modoEdicao} ?
@{/games/{id}/editar(id=${game.id})} : @{/games}"
4     th:object="${game}"
5     method="post">
6
7     <input type="hidden" th:field="*{id}">
8
9     <input id="nome" type="text" th:field="*{nome}">
10    <input id="plataforma" type="text"
th:field="*{plataforma}">
11    <input id="genero" type="text" th:field="*{genero}">
12
13    <select id="status" th:field="*{status}">
14        <option value="">Selecione</option>
15        <option value="Quero jogar">Quero jogar</option>
16        <option value="Jogando">Jogando</option>
17        <option value="Zerado">Zerado</option>
18    </select>
19 </form>
```

**th:object="\${game}"**

Define que o formulário inteiro trabalha com um objeto Game.

**th:field="\*{...}"**

Liga cada campo ao atributo correspondente do objeto. Menos código manual, menos chance de erro.

**Ação dinâmica**

O mesmo formulário atende cadastro e edição. Quem decide é a flag modoEdicao.

**Input hidden id**

Na edição, o id acompanha o objeto e mantém o vínculo com o registro correto.

## Validação e feedback visual na tela

### erros de campo no formulário

```
1 <small class="erro"
2
3 th:if="${#fields.hasErrors('nome')}}"
4   th:errors="*{nome}"></small>
5 <small class="erro"
6
7 th:if="${#fields.hasErrors('plataforma')}}"
8   th:errors="*{plataforma}"></small>
9 <small class="erro"
10
11 th:if="${#fields.hasErrors('status')}}"
12   th:errors="*{status}"></small>
```

### mensagem de sucesso na listagem

```
1 <div th:if="${mensagem}"
2   class="mensagem-sucesso"
3   th:text="${mensagem}"></div>
```

### Ligação com o Controller

Quando `BindingResult` tem erro, o método retorna formulario novamente. O Thymeleaf reaproveita o objeto e mostra os avisos.

### `#fields.hasErrors`

Verifica se o campo específico falhou na validação.

### `th:errors`

Escreve a mensagem definida nas anotações da entidade `Game`.

### Flash attribute

Depois de salvar, atualizar ou excluir, a tela de lista pode exibir uma confirmação amigável.

## style.css: separando estrutura de apresentação

style.css

```
1 body {
2     background: #f3f5fb;
3     color: #1f2937;
4 }
5
6 .card {
7     background: #ffffff;
8     border-radius: 14px;
9     box-shadow: 0 10px 25px rgba(15, 23, 42,
10 0.08);
11     padding: 24px;
12 }
13 button, .botao-principal {
14     background: #2563eb;
15     color: #ffffff;
16 }
17
18 .badge {
19     background: #dbeafe;
20     color: #1d4ed8;
21 }
22
23 @media (max-width: 768px) {
24     table { min-width: 720px; }
25 }
```

### Consistência visual

container, card, botões e badge mantêm a mesma identidade em todas as páginas.

### Melhor experiência

Cores, espaçamento e sombra fazem o projeto parecer produto de verdade, não dever de casa triste.

### Responsividade básica

O media query ajuda a página a degradar melhor em telas menores.

### Mensagem importante

Thymeleaf cuida da renderização dinâmica. CSS cuida da aparência. Cada um no seu quadrado e a aplicação agradece.

# Fluxo da View e roteiro de implementação em sala

## Fluxo completo da renderização

- 1. Controller** retorna "lista" ou "formulario" e envia atributos ao Model
- 2. Thymeleaf** lê os atributos e processa th:text, th:each, th:field e afins
- 3. HTML final** o navegador recebe a página já montada
- 4. CSS** o arquivo style.css dá acabamento visual à resposta
- 5. Usuário** interage novamente e inicia outro ciclo de requisição

## Roteiro curto para ensinar ao vivo

### 1 Montar lista.html

Criar cabeçalho, card e tabela. Depois substituir conteúdo fixo por th:each e th:text.

### 2 Montar formulario.html

Criar o form, ligar com th:object e mapear os campos com th:field.

### 3 Adicionar erros e feedback

Mostrar th:errors no form e mensagem de sucesso na listagem.

### 4 Aplicar CSS

Apresentar style.css como camada separada e explicar o ganho de manutenção.

## Checklist da explicação

- de onde vem o atributo games
- como o objeto game entra no formulário
- por que o mesmo HTML serve para cadastro e edição
- onde o CSS é carregado